Zaawansowane techniki programowania

Laboratorium 4

ZMIENDAK WIKTOR

ZADANIA OBOWIĄZKOWE:

1. Tworzymy case dla rabatu SAVE10 który odejmuje 10% od wartości subtotal.

```
public BigDecimal calculateDiscount(Product product, int quantity, String discountCode) {
    String code = normalizeCode(discountCode);
    BigDecimal sub = subtotal(product, quantity);

    switch (code) {
        case "SAVE10":
            return percent(sub, SAVE10);
```

```
private BigDecimal percent(BigDecimal value, BigDecimal percent) {
    return value.multiply(percent).setScale(newScale: 2, RoundingMode.HALF_UP);
}
```

W innym przypadku zwracamy 0.

```
default:
    return ZERO;
```

2. Weryfikujemy czy quantity jest większe od 0. Pobieramy produkt a następnie sprawdzamy stan magazynu.

```
public ShopOrder placeOrder(OrderDraft draft) {
    // Validation phase (all checks before any state modification)
    if (draft.getQuantity() <= 0) {
        throw new IllegalArgumentException(s: "Quantity must be greater than 0");
    }

    Product product = productRepository.findById(draft.getProductId())
            .orElseThrow(() -> new NoSuchElementException(s: "Product not found"));

    if (product.getStock() < draft.getQuantity()) {
        throw new IllegalStateException(s: "Insufficient stock");
    }
```

Obliczamy subtotal oraz total

```
// Calculation phase
BigDecimal subtotalAmount = subtotal(product, draft.getQuantity());
BigDecimal discount = pricingPolicy.calculateDiscount(product, draft.getQuantity()
BigDecimal subtotalAfterDiscount = subtotalAmount.subtract(discount);
BigDecimal shipping = pricingPolicy.calculateShipping(subtotalAfterDiscount);
BigDecimal total = subtotalAfterDiscount.add(shipping);
int loyaltyPoints = pricingPolicy.calculateLoyaltyPoints(total);
Instant createdAt = now();
```

Zmniejszamy stock produktu oraz zapisujemy go wraz z zamówieniem do repozytorium

```
// State modification phase
product.decreaseStock(draft.getQuantity());
productRepository.save(product);

ShopOrder order = new ShopOrder(
        id: null,
        draft.getProductId(),
        draft.getQuantity(),
        subtotalAmount,
        discount,
        shipping,
        total,
        loyaltyPoints,
        createdAt);

return shopOrderRepository.save(order);
```

ZADANIA DODATKOWE:

1. Rozszerzamy mechanizm rabatów o wersję BULK5 oraz VIP. Dodajemy również mechanizm darmowej dostawy oraz punktów lojalnościowych.

```
private static final BigDecimal ZERO = money(value: "0.00");
private static final BigDecimal DELIVERY_COST = money(value: "15.00");
private static final BigDecimal FREE_SHIPPING_THRESHOLD = money(value: "300.00");
private static final BigDecimal SAVE10 = money(value: "0.10");
private static final BigDecimal BULK5 = money(value: "0.05");
private static final BigDecimal VIP = money(value: "0.15");

@Override
public BigDecimal calculateDiscount(Product product, int quantity, String discountCode)
    String code = normalizeCode(discountCode);
    BigDecimal sub = subtotal(product, quantity);

    switch (code) {
        case "SAVE10":
            return percent(sub, SAVE10);
        case "BULK5":
            if (quantity >= 5) {
                return percent(sub, BULK5);
            }
            return ZERO;
        case "VIP":
            return percent(sub, VIP);
        default:
            return ZERO;
    }
}
```

2. Rozszerzamy ShopServiceImpl o mierzenie czasu.

```
Instant createdAt = now();
```

```
private Instant now() {
    return Instant.now(clock);
}
```

W przypadku błędnego zamówienia nic nie zostaje zapisane. Służą do tego utworzone walidatory

```
// Validation phase (all checks before any state modification)
if (draft.getQuantity() <= 0) {
    throw new IllegalArgumentException(s: "Quantity must be greater than 0");
}

Product product = productRepository.findById(draft.getProductId())
        .orElseThrow(() -> new NoSuchElementException(s: "Product not found"));

if (product.getStock() < draft.getQuantity()) {
    throw new IllegalStateException(s: "Insufficient stock");
}
```

WYNIKI TESTÓW:

```
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.219 s -- in pl.example.lab4.Lab4Grade3UnitTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  11.411 s
[INFO] Finished at: 2026-03-24T20:35:23+01:00
[INFO] ------------------------------------------------------------------------
```

```
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap class
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.727 s -- in pl.example.lab4.Lab4Grade4UnitT
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.836 s
[INFO] Finished at: 2026-03-24T20:36:09+01:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\ziggs\Desktop\Studies\Cybersecurity\Semestr_I\ZTP\Zmiendak_Wiktor_LAB4\Zmiendak_Wiktor_LAB4>
```